

Planetaire Mono

Bers l'or f-m
a C r i f a f C m r
a r e i C

i . C a y
/ r l a a i r
i o . x . o

la a i l'or f-m a C m r i m r a f f t t a t m r a t t i l y
l i l l'or f-m f Bers, a y a a i d a y i t a i l o a i d
f r i f C C r i f l a y, a a r e i C m r l a
i f a f C r : f f a t i, a y m l, a x s, o o o t a l r i C.

l r a l i a f f f i m i r C a r C l a i y a a i f f t t a t
C m i a i a i a a a a C l a i y a f f l l f f a m m i
l a a a a a . a a m i a a a a i B e r s f m a r t t a
a a a.

Planetaire

Planetaire (400)

Planetaire (400)

Medium (400)

Medium Italic (400)

SemiBold (800)

SemiBold Italic (800)

Planetaire (700)

Planetaire (700)

EXTRA BOLD (900)

EXTRA BOLD ITALIC (900)

Planetaire

Planetaire f-m Bers

Planetaire f-m Bers (f-m Bers)

Planetaire f-m Bers, f-m Bers, f-m Bers

Planetaire f-m Bers

Planetaire f-m Bers

Planetaire f-m Bers

Planetaire f-m Bers

የኢትዮጵያ ፌዴራላዊ ዲሞክራሲያዊ ሪፐብሊክ

placental transfer variability, fetal Bcrs' titration and/or fetal
immunity and/or fetal variability at small size and in low-litter-size displays.

1. 7-10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100. 101. 102. 103. 104. 105. 106. 107. 108. 109. 110. 111. 112. 113. 114. 115. 116. 117. 118. 119. 120. 121. 122. 123. 124. 125. 126. 127. 128. 129. 130. 131. 132. 133. 134. 135. 136. 137. 138. 139. 140. 141. 142. 143. 144. 145. 146. 147. 148. 149. 150. 151. 152. 153. 154. 155. 156. 157. 158. 159. 160. 161. 162. 163. 164. 165. 166. 167. 168. 169. 170. 171. 172. 173. 174. 175. 176. 177. 178. 179. 180. 181. 182. 183. 184. 185. 186. 187. 188. 189. 190. 191. 192. 193. 194. 195. 196. 197. 198. 199. 200. 201. 202. 203. 204. 205. 206. 207. 208. 209. 210. 211. 212. 213. 214. 215. 216. 217. 218. 219. 220. 221. 222. 223. 224. 225. 226. 227. 228. 229. 230. 231. 232. 233. 234. 235. 236. 237. 238. 239. 240. 241. 242. 243. 244. 245. 246. 247. 248. 249. 250. 251. 252. 253. 254. 255. 256. 257. 258. 259. 260. 261. 262. 263. 264. 265. 266. 267. 268. 269. 270. 271. 272. 273. 274. 275. 276. 277. 278. 279. 280. 281. 282. 283. 284. 285. 286. 287. 288. 289. 290. 291. 292. 293. 294. 295. 296. 297. 298. 299. 300. 301. 302. 303. 304. 305. 306. 307. 308. 309. 310. 311. 312. 313. 314. 315. 316. 317. 318. 319. 320. 321. 322. 323. 324. 325. 326. 327. 328. 329. 330. 331. 332. 333. 334. 335. 336. 337. 338. 339. 340. 341. 342. 343. 344. 345. 346. 347. 348. 349. 350. 351. 352. 353. 354. 355. 356. 357. 358. 359. 360. 361. 362. 363. 364. 365. 366. 367. 368. 369. 370. 371. 372. 373. 374. 375. 376. 377. 378. 379. 380. 381. 382. 383. 384. 385. 386. 387. 388. 389. 390. 391. 392. 393. 394. 395. 396. 397. 398. 399. 400. 401. 402. 403. 404. 405. 406. 407. 408. 409. 410. 411. 412. 413. 414. 415. 416. 417. 418. 419. 420. 421. 422. 423. 424. 425. 426. 427. 428. 429. 430. 431. 432. 433. 434. 435. 436. 437. 438. 439. 440. 441. 442. 443. 444. 445. 446. 447. 448. 449. 450. 451. 452. 453. 454. 455. 456. 457. 458. 459. 460. 461. 462. 463. 464. 465. 466. 467. 468. 469. 470. 471. 472. 473. 474. 475. 476. 477. 478. 479. 480. 481. 482. 483. 484. 485. 486. 487. 488. 489. 490. 491. 492. 493. 494. 495. 496. 497. 498. 499. 500. 501. 502. 503. 504. 505. 506. 507. 508. 509. 510. 511. 512. 513. 514. 515. 516. 517. 518. 519. 520. 521. 522. 523. 524. 525. 526. 527. 528. 529. 530. 531. 532. 533. 534. 535. 536. 537. 538. 539. 540. 541. 542. 543. 544. 545. 546. 547. 548. 549. 550. 551. 552. 553. 554. 555. 556. 557. 558. 559. 560. 561. 562. 563. 564. 565. 566. 567. 568. 569. 570. 571. 572. 573. 574. 575. 576. 577. 578. 579. 580. 581. 582. 583. 584. 585. 586. 587. 588. 589. 590. 591. 592. 593. 594. 595. 596. 597. 598. 599. 600. 601. 602. 603. 604. 605. 606. 607. 608. 609. 610. 611. 612. 613. 614. 615. 616. 617. 618. 619. 620. 621. 622. 623. 624. 625. 626. 627. 628. 629. 630. 631. 632. 633. 634. 635. 636. 637. 638. 639. 640. 641. 642. 643. 644. 645. 646. 647. 648. 649. 650. 651. 652. 653. 654. 655. 656. 657. 658. 659. 660. 661. 662. 663. 664. 665. 666. 667. 668. 669. 670. 671. 672. 673. 674. 675. 676. 677. 678. 679. 680. 681. 682. 683. 684. 685. 686. 687. 688. 689. 690. 691. 692. 693. 694. 695. 696. 697. 698. 699. 700. 701. 702. 703. 704. 705. 706. 707. 708. 709. 710. 711. 712. 713. 714. 715. 716. 717. 718. 719. 720. 721. 722. 723. 724. 725. 726. 727. 728. 729. 730. 731. 732. 733. 734. 735. 736. 737. 738. 739. 740. 741. 742. 743. 744. 745. 746. 747. 748. 749. 750. 751. 752. 753. 754. 755. 756. 757. 758. 759. 760. 761. 762. 763. 764. 765. 766. 767. 768. 769. 770. 771. 772. 773. 774. 775. 776. 777. 778. 779. 780. 781. 782. 783. 784. 785. 786. 787. 788. 789. 790. 791. 792. 793. 794. 795. 796. 797. 798. 799. 800. 801. 802. 803. 804. 805. 806. 807. 808. 809. 810. 811. 812. 813. 814. 815. 816. 817. 818. 819. 820. 821. 822. 823. 824. 825. 826. 827. 828. 829. 830. 831. 832. 833. 834. 835. 836. 837. 838. 839. 840. 841. 842. 843. 844. 8

[illegible]
$$E_1 E_2 C \cdot C E_1 K I Z \cdot Z E_1 K I Z \cdot I U_1 K I Z$$

r-ai-f di-tal a-i-f r-flam a-r-yerl-yo-.kal
n-l-x-l-m-i-t-jäl t-a-dl-worl. El-el-
w-cilla-ti-t-mia feli-a-ill-y-i-i. bil-amalı ta-ta
yağı-ş-f-re-ça-t-at-t-e-i.

CREEK

Ξεσπάζω τὴν ψυχ-φθόρα βδελυγμία.

CAIRGIC

Съешь же ещё этих мягких французских булок, да выпей чаю. Широкая электрификация южных губерний даст мощный толчок подъёму сельского хозяйства.

$$p_{\text{la-0+ai}}^{\text{ro}} \text{ --- } p_{\text{it+it}}^{\text{C-m/ll}} / p_{\text{la-0+ai}}^{\text{ro}} \text{ --- } 0.67-0.7$$

ATTN: INBIC - "COMBINE MESSAGES AND INTERFERENCE" (72-0)

БЕЗ П. ЗДЕ/Е СЛОКЕБ, ПСГ. 7 ПУБЛИК. РАД.

$\sum_{i=1}^n \frac{1}{r_i} = r - C_0$

$\gamma_{tr}^{OGUC} \gamma_{tr}^{ON}$

x $\downarrow$ z U_{mm} $3r$ y 0_r t 10 x_{HP} z 0_r t 10 $3r$ y

[illegible][illegible]

blatn' ai r' C-m/110 y / r' ai r' o e r - x . x

PyTorch vs GPT: The Complete Algorithm

The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os      # os.path.exists
import math    # math.log, math.exp
import random  # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset 'docs': List[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
print(f"num docs: {len(docs)}")

# Let there be a Tokenizer to translate strings to sequences of integers ("tokens") and back
uchars = sorted(set(''.join(docs))) # unique characters in the dataset become token ids 0..n-1
BOS = len(uchars) # token id for a special Beginning of Sequence (BOS) token
vocab_size = len(uchars) + 1 # total number of unique tokens, +1 is for BOS
print(f"vocab size: {vocab_size}")

# Let there be Autograd to recursively apply the chain rule through a computation graph
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads') # Python optimization for memory usage

    def __init__(self, data, children=(), local_grads=()):
        self.data = data # scalar value of this node calculated during forward pass
        self.grad = 0 # derivative of the loss w.r.t. this node, calculated in backward pass
        self._children = children # children of this node in the computation graph
        self._local_grads = local_grads # local derivative of this node w.r.t. its children

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self): return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self): return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
    def __neg__(self): return self * -1
    def __add__(self, other): return self + other
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return other + (-self)
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other**-1
    def __rtruediv__(self, other): return other * self**-1

    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._children:
                    build_topo(child)
            topo.append(v)
        build_topo(self)
        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._local_grads):
                child.grad += local_grad * v.grad

# Initialize the parameters, to store the knowledge of the model
n_layer = 1 # depth of the transformer neural network (number of layers)
n_embd = 16 # width of the network (embedding dimension)
block_size = 16 # maximum context length of the attention window (note: the longest name is 15 characters)
n_head = 4 # number of attention heads
head_dim = n_embd // n_head # derived dimension of each head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std)) for _ in range(nin)] for _ in range(nout)]
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd), 'lm_head': matrix(vocab_size, n_embd)}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
```

LaTeX-style text at the bottom of the page.

```

state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
params = [p for mat in state_dict.values() for row in mat for p in row] # flatten params into a single list[Value]
print(f'num params: {len(params)}')

# Define the model architecture: a function mapping tokens and parameters to logits over what comes next
# Follow GPT-2, blessed among the GPTs, with minor differences: layernorm -> rmsnorm, no biases, GeLU -> ReLU
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(w, x)) for wo in w]

def softmax(logits):
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]

def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]

def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id] # token embedding
    pos_emb = state_dict['wpe'][pos_id] # position embedding
    x = [t + p for t, p in zip(tok_emb, pos_emb)] # joint token and position embedding
    x = rmsnorm(x) # note: not redundant due to backward pass via the residual connection

    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.5 for t in range(len(k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h))) for j in range(head_dim)]
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
        # 2) MLP block
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits

# Let there be Adam, the blessed optimizer and its buffers
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params) # first moment buffer
v = [0.0] * len(params) # second moment buffer

# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):
    # Take single document, tokenize it, surround it with BOS special token on both sides
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)

    # Forward the token sequence through the model, building up the computation graph all the way to the loss
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document sequence. May yours be low.

    # Backward the loss, calculating the gradients with respect to all model parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the corresponding gradients
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))

```

```

p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
p.grad = 0

print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}", end='\r')

# Inference: may the model babble back to us
temperature = 0.5 # in (0, 1], control the "creativity" of generated text, low to high
print("\n--- inference (new, hallucinated names) ---")
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
    print(f"sample {sample_idx+1:2d}: {''.join(sample)}")

```


РҮҢӨТҮРӨ СҮРҮЦӨРӨ

БҮЗІС ГҮҮХ НҮҮЕҮСҮЭ: ҮҮ-м БҮҮС

♦ B C D E F G H I J K L M N O P Q R S T U V W X Y Z

БҮЗІС ГҮҮХ ГҮҮЕҮСҮЭ: ҮҮ-м БҮҮС

a b c d e f g h i j k l m n o p q r s t u v w x y z

ОХИЛЗ: Ө-ү ҮҮ-м БҮҮС (А-а, Б-б, В-в, Г-г, Д-д, Е-е, Ё-ё, З-з, И-и, Й-й, К-к, Л-л, М-м, Н-н, О-о, П-п, Р-р, С-с, Т-т, У-у, Ф-ф, Х-х, Ц-ц, Ч-ч, Ш-ш, Щ-щ, Ъ-ь, Ы-ы, Ь-ь, Э-э, Ю-ю, Я-я)

0 1 2 3 4 5 6 7 8 9

БҮЗІС ГҮҮХ НҮҮЕҮСҮЭ: ҮҮ-м БҮҮС

! " # \$ % & ' () * + , - . / : ; < = > ? @ [\] ^ _
{ | } ~

EXTRADED ГҮҮХ: ҮҮ-м БҮҮС

À Á Â Ã Ä Å Æ Ç È É Ê Ë Ì Í Î Ï Ð Ñ Ò Ó Ô Õ Ö Ø Ù Ú Û Ü Ý Þ ß à á â ã ä å æ ç è é ê ë ì í î ï ð ñ ò ó ô õ ö ø ù ú û ü ý þ ÿ

Ā ā Ă ă Ą ą Ć ć Ĉ ĉ Ċ ċ Č č Ď ě Đ đ Ē ē Ĕ ĕ Ė ė Ę ę Ğ ğ

Ğ ğ Ġ ġ Ģ ģ Ĥ ĥ Ħ ħ Ĩ ĩ Ī ī Ĭ ĭ Ĵ ĵ Ķ ķ

СҮҮЕҮСҮЭ: ҮҮ-м БҮҮС

♦ B C D E F G H I J K L M N O P Q R S T U V W X Y Z

α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω

СҮҮЕҮСҮЭ: ҮҮ-м БҮҮС

АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ

абвгдежзийклмнопрстуфхцчшщъыьэюя

Printable ASCII Combinations

ASCII (00)

The quick brown fox jumps over the lazy dog
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

ASCII (00)

The quick brown fox jumps over the lazy dog
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

ASCII (00)

The quick brown fox jumps over the lazy dog
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

ASCII ASCII (00)

The quick brown fox jumps over the lazy dog
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

ASCII (00)

The quick brown fox jumps over the lazy dog
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

ASCII ASCII (00)

The quick brown fox jumps over the lazy dog
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

BORD (700)

The quick brown fox jumps over the lazy dog
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

BORD ASCII (700)

The quick brown fox jumps over the lazy dog
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

EXASCII (000)

The quick brown fox jumps over the lazy dog

01_345678, A9pCC-Q {[(>)]} !@#\$\$%

exitBord ilric (000)

TLc QYfCk bLOWL jOx jYm-a OvCL ALc 'ozy dog

01_345678, A0bCC-Q {[(>)]} !@#\$\$%

SIZE COMPIZ0: BEC0F0 1 / IONZ SIZEZ

1 7-iCk LF-Wn f-- l-mrK --Qr +kQ la-Y J--

1 7-iCk LF-Wn f-- l-mrK --Qr +kQ la-Y J--

1 7-iCk LF-Wn f-- l-mrK --Qr +kQ la-Y J--

1 7-iCk LF-Wn f-- l-mrK --Qr +kQ la-Y J--

1 7-iCk LF-Wn f-- l-mrK --Qr +kQ la-Y J--

რეგულაციები

საერთაშორისო სტანდარტები

I 1 | რეგულაციები . ლ-რეგულაციები . აივით . რიგ

0 0 0 რეგულაციები . აივით . ლ-რეგულაციები -

r n m რ . . m - ცილა რეგულაციები აივით-ი-ბერს

5 S 8 B აივით . ლ . აივით . ლ B

2 Z 6 G აივით . ლ . აივით . ლ c

; : მიმართულია ლ-რეგულაციები - აივით-ი-ბერს აივით-ი-ბერს

() { } [] რეგულაციები ლ-რეგულაციები ლ-რეგულაციები

- - - რეგულაციები-მიმართულია ლ-რეგულაციები - რეგულაციები - რეგულაციები

“ ” ” რეგულაციები-მიმართულია რეგულაციები-მიმართულია რეგულაციები-მიმართულია რეგულაციები-მიმართულია

‘ ’ ‘ ’ რეგულაციები-მიმართულია რეგულაციები-მიმართულია რეგულაციები-მიმართულია რეგულაციები-მიმართულია

საერთაშორისო სტანდარტები (რეგულაციები)

რეგულაციები (რეგულაციები)

რეგულაციები / რეგულაციები (რეგულაციები)

0 0 0 -

რეგულაციები რეგულაციები რეგულაციები

0 0 0 -

რეგულაციები რეგულაციები რეგულაციები

საერთაშორისო სტანდარტები

() { } [] < > « » < >

საერთაშორისო სტანდარტები

+ - * / = ≠ ≤ ≥ ± × ÷ → ← ↑ ↓

ლა-რეგულაციები რეგულაციები-მიმართულია რეგულაციები-მიმართულია რეგულაციები-მიმართულია

$$|a_{i+1}|^2 \leq \frac{C_m |a_i|^2}{|a_i|^2 + 1} \quad \text{for } i \geq 1.$$

ERG on x conc

$\frac{d}{dt} \left(\int_{\Omega} u^2 dx + \int_{\Gamma} u^2 dS \right) = -2 \int_{\Omega} u \Delta u dx - 2 \int_{\Gamma} u \nabla_n u dS$

NI 

LIFE TYPE



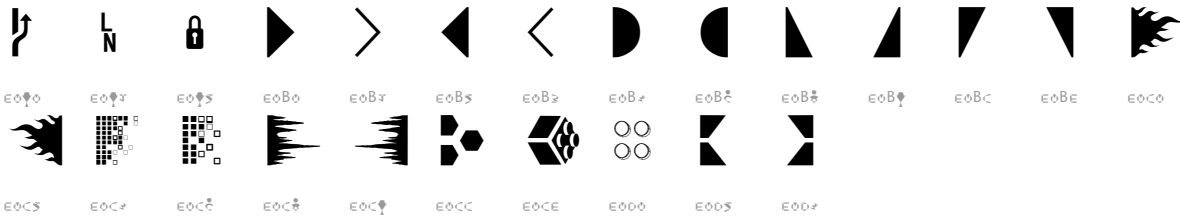
DE/EGOMEDIA



▲◆⊥⊕◆



BOERHIE


$$p_{la^{-0}ai^0} = \frac{1}{2} \left(1 + \frac{C_{m/l}^0}{p_{la^{-0}ai^0}} \right) \cdot 0.675$$

Provenance & Conce

Source code

612 μono	o i d i l o f i l a r t i m i l a r l i l i t i c u r i i l l a y a . l a t a i t a l l a t t (t , a) , i i i o o , a n d r a t i / f o o / 6 y i l l i c o l y r a f m b e r s t h e o r d y r f o c i o a c o n t o l c i r c l e t o o i d f l y i g a m t i a t i w i t h c i r c l e (f a l t) a n d r o c t a d l (o r) v a i a t a .
3C	c r i a i m p i a , y o c o o c i d o f r a r c o c o c r i i l l a t a r f e n t a r c t r o : f r c t a t i n , y m t l a , a n d v o r a l l m a r i c a .
6 μo n t	y a o x c r t y r o , i c n r a c i n d f i o c . r s , o o a t o l l r i c i c l i i d w o r l i o , e n t w a t m , o i c r a , m a r i a l o o i d , a n m r o , a l l i n l l a v i a t t a a e n t t a a .

ህዲህ ምዕራብ

[illegible]

cence

placat ai a fost înlocuit cu placat nou, iar o denonțanță 1.1 (OAR-1.1).

the over-all wage reduction, modification, and redistribution of rates, including in commercial practice, provided that modifications are not a result of the market and carry a different name.

1. The Court will carry out the following:

- $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \Delta u dx = - \int_{\mathbb{R}^n} |\nabla u|^2 dx \leq 0$
- $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \Delta u dx = - \int_{\mathbb{R}^n} |\nabla u|^2 dx \leq 0$
- $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \Delta u dx = - \int_{\mathbb{R}^n} |\nabla u|^2 dx \leq 0$